Flagpost

USER GUIDE

Judge Guide

Authoring and running a competition on Flagpost

For **Judges & organisers** · Flagpost **1.8.0-src** · 19 October 2026

Contents

1	The judge role	3
2	Authoring challenges	5
3	Programming problems & judging	7
4	Challenge instances	10
5	Configuring & running the event	11
6	Supporting competitors	13
7	Watching the event	15
8	FAQ	17

CHAPTER 1

The judge role

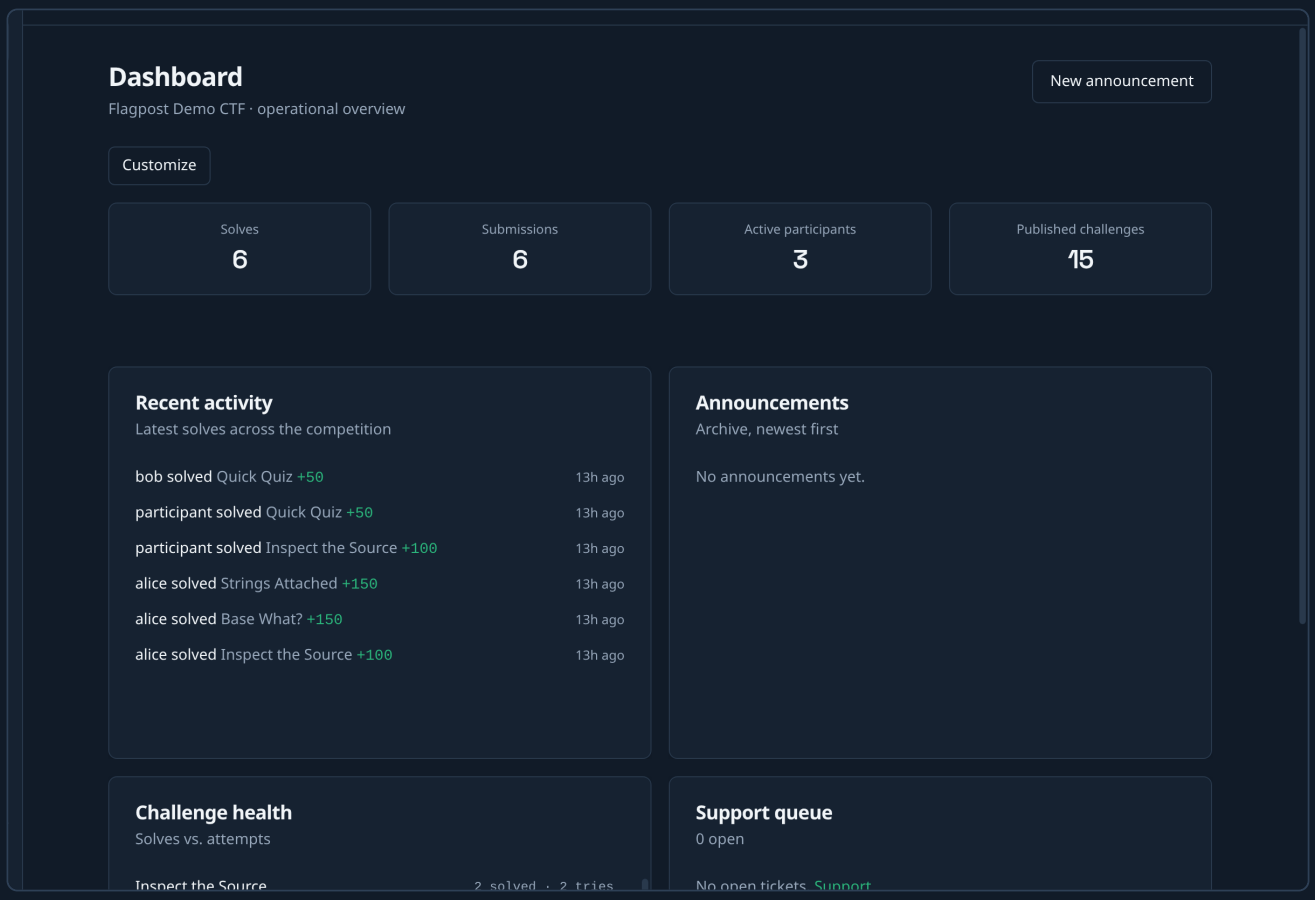
A judge is **competition-scoped staff**: everything in this guide applies to the competition(s) you help organise, granted through your role there. Site-wide concerns — user accounts, sign-in options, branding, the judge and instance infrastructure — belong to administrators, who have their own guide. This one is a click away in the app, under **User guides** in the top bar.

What your role gives you that competitors don't see:

- **Manage buttons** on the surfaces you can author — most importantly Manage challenges on the challenges page.
- **Drafts** — unpublished challenges are visible to you everywhere, labelled, and invisible to competitors.
- **The status-gate bypass** — before the competition starts (and after it ends) competitors see a closed notice, while anyone who can edit challenges keeps full access. While the event is paused, competitors can't submit but you still can.

> **YOUR SOLVES ARE REAL**
>
> A staff flag submission is graded like anyone's: in individual mode it isn't ranked, but it takes **first blood**, decays a dynamic challenge's value and fires automations; in team mode you'd have to join a team first. Open and download freely on the live board, but rehearse flag submission in a clone — and for programming problems use **Run examples** or the reference check, which never score.

Your home: the dashboard



The operational overview: At a glance, Recent solves, the Announcements archive, Challenge health and the Support queue — every widget live.

The header carries **New announcement** and **Customize**, which opens an edit mode: drag to move or resize, **Add section** to pick from a catalog (unsolved challenges, difficulty progress, team activity, brute-force watch, a moderation feed, schedule & status, instance health), then **Save layout**. Until your first challenge is published, a Getting started card walks the first steps.

Permissions are granular

Your role may be narrower than the built-in Judge. If a surface is missing for you, this is what gates it:

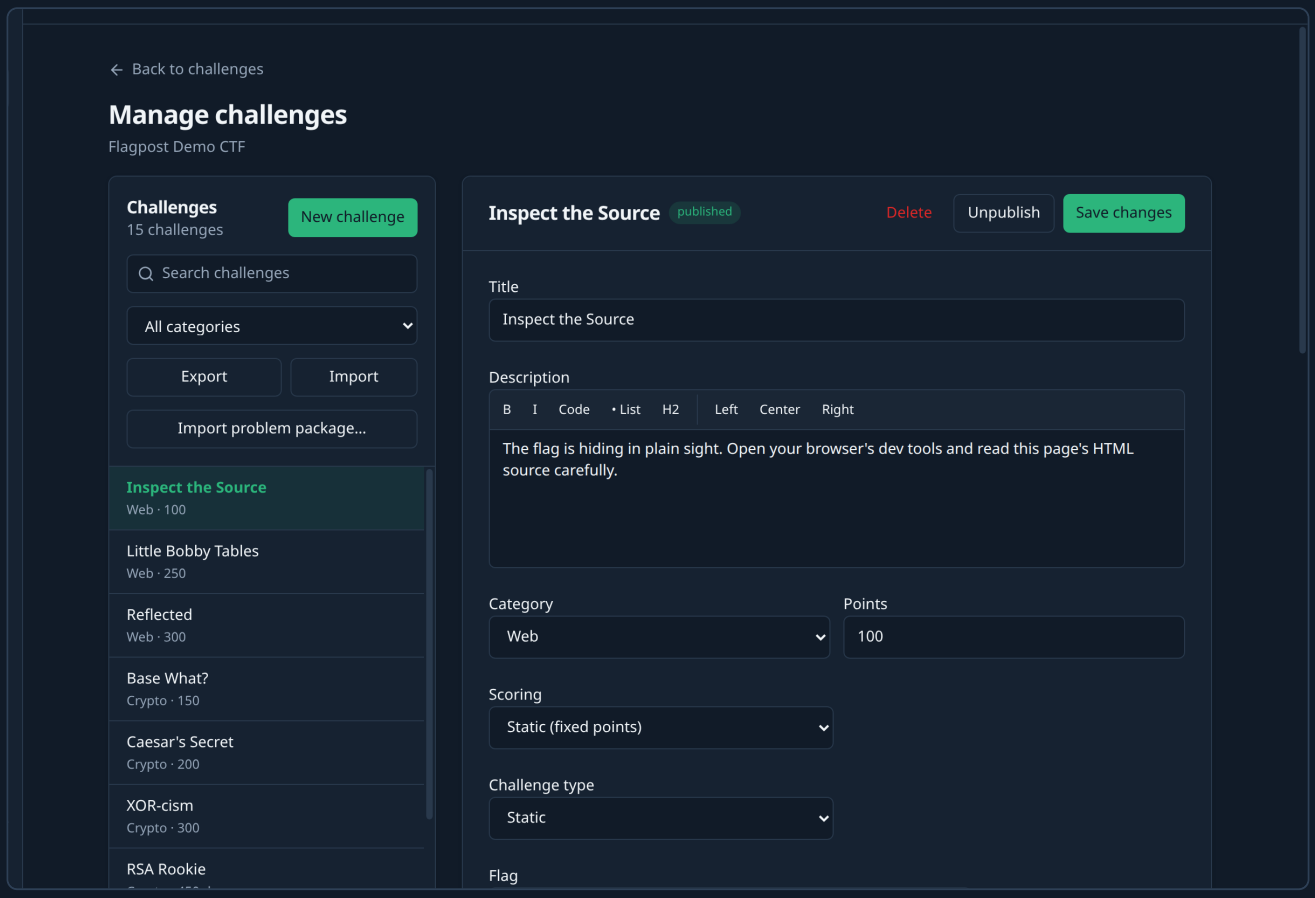
SURFACE	GRANT
Settings → Modules / Sharing	manage modules / manage invite links
Settings → Certificates / Reports	manage certificates / generate report
Start / Stop · Pause · Freeze	manage schedule · edit competition · scoreboard freeze
Sidebar: Judging · Instances · AI transcripts	judging view · instance view · AI transcripts
Participants → Create award	score override

Ask your administrator which grants your role carries.

CHAPTER 2

Authoring challenges

Open **Manage challenges** from the challenges page: your challenge list on the left, the editor for the selected challenge on the right.



The list pane searches and filters by category, and carries New challenge, Export and Import (plus problem-package import when the code judge is on); the editor keeps Save, Publish and Delete together above the fields.

The lifecycle: draft → published

New challenges are **drafts** — invisible to competitors, so you can build openly during the event. Each type has its own publish requirement: a static or regex challenge needs a flag, multiple choice a correct option, a programming problem at least one test case. Unpublishing hides a challenge again without deleting anything; deleting removes it and its attachments, hints and solve history.

What a challenge can carry

- **Challenge type** — fixed once created. Static (exact match, with a **Case-insensitive flag** option), Regex (pattern-graded), Multiple choice (up to ten options, one marked correct; pairs with guess caps), or Programming (solved by submitting code — chapter 3).
- **Scoring** — Static (fixed points) or Dynamic (decays as more solve): the value falls from the initial points toward a minimum over roughly the stated number of solves. Every solver holds the current value.

- **More options** (collapsed) — **Connection info** (`nc host 1337`, a URL; shown with a copy button once unlocked), **Release at** (a published challenge stays hidden until then — waves with no one awake to push a button), **Prerequisites** (an unlock chain), **Difficulty** and **Tags** (once defined in Settings → Challenges).

The flag itself is **write-only**: the editor shows that one is set, never its value — so a shared screen never leaks it.

After creating

Create challenge first; the sections that need a saved challenge appear below it: **Attachments**, **Hints** (free or costing points, optionally Hidden until released at a set time), **Guesses** (multiple choice — chapter 6), **Deployment** (instanced challenges — chapter 4) and **Test cases** (programming — chapter 3).

Categories, difficulty & tags

Settings → Challenges owns the vocabulary: categories (the chips competitors filter by), difficulty tiers, and tags. Set them early — consistent tagging is what makes a 40-challenge board navigable.

Import & export

Export downloads the set as a ctfcli-format zip and **Import** loads one (challenges whose title already exists are skipped, never overwritten). Descriptions, regex patterns, hints, attachments, prerequisites and deployments round-trip; **static flags do not** — they're stored hashed — so keep flags in your own source and re-add them after an import. Multiple-choice options aren't carried either, and programming problems export separately as problem packages (chapter 3).

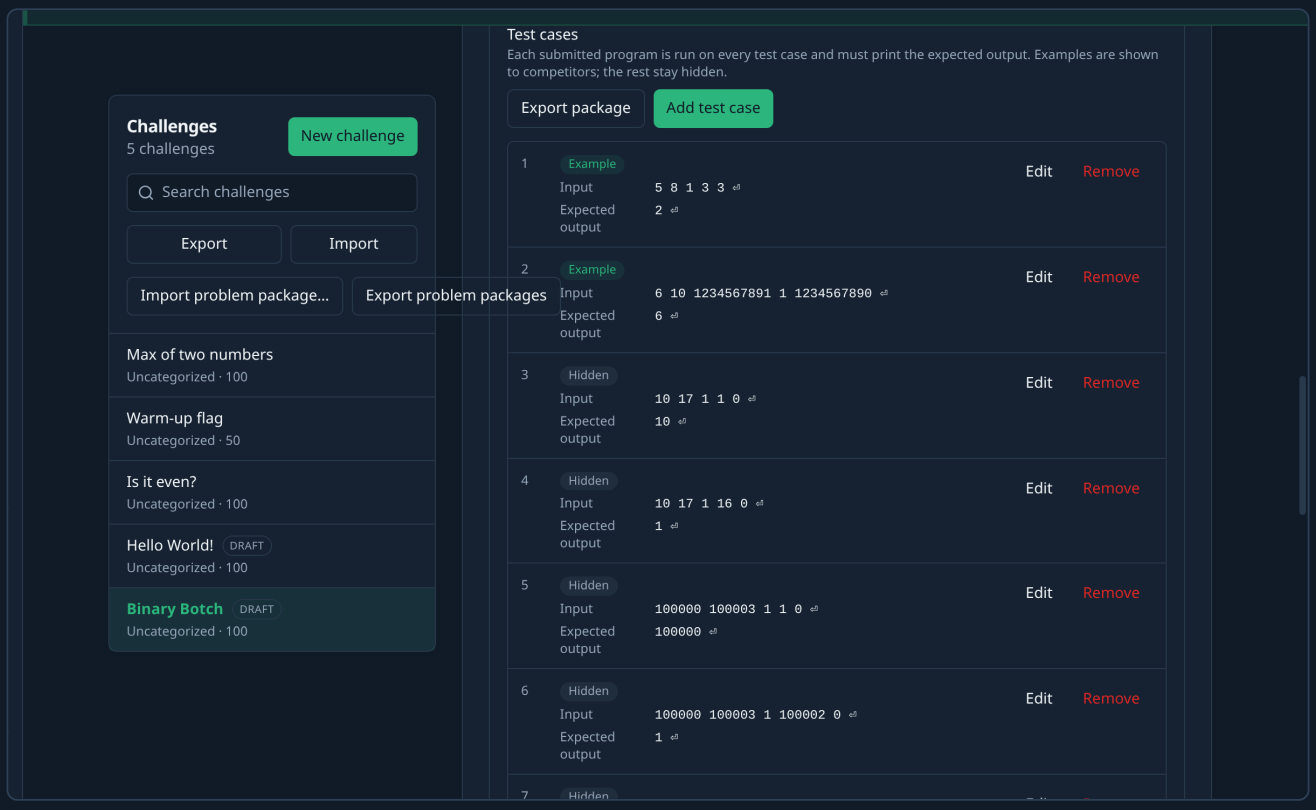
CHAPTER 3

Programming problems & judging

A **programming problem** is solved by submitting a program rather than a flag — the ICPC-style problem: read standard input, print the answer. It's available once an administrator has set up the **code judge** (Admin → Site settings → Code judge) and the competition's **Code judge** module is on.

Authoring a problem

Choose the challenge type Programming (solved by submitting code), write the statement in the description, and **Create challenge**. The **Test cases** section then appears.



Each submitted program runs on every test case. Examples are shown with the statement; hidden cases are only used for judging.

- **Add test case** — Type or paste an input and its expected output, or Upload files for large or binary data. Tick **Show to competitors as an example** for the ones that illustrate the problem; keep at least one hidden, or a program could just print the answers.
- **Advanced** — per-problem **time**, **memory** and **output limits** (blank uses the site default), **how output is checked** (Ignore extra spaces and blank lines, Must match exactly, or Numbers may differ slightly), **Ignore upper/lower case**, and which **Languages** are allowed.
- **Check with a reference solution** — run a solution you know is right against every test, through the real judge. It reports each test, never scores or blocks publishing, and tells you when the tests or limits have changed since. Make it a habit: it catches a wrong expected output or a too-tight limit before competitors do.

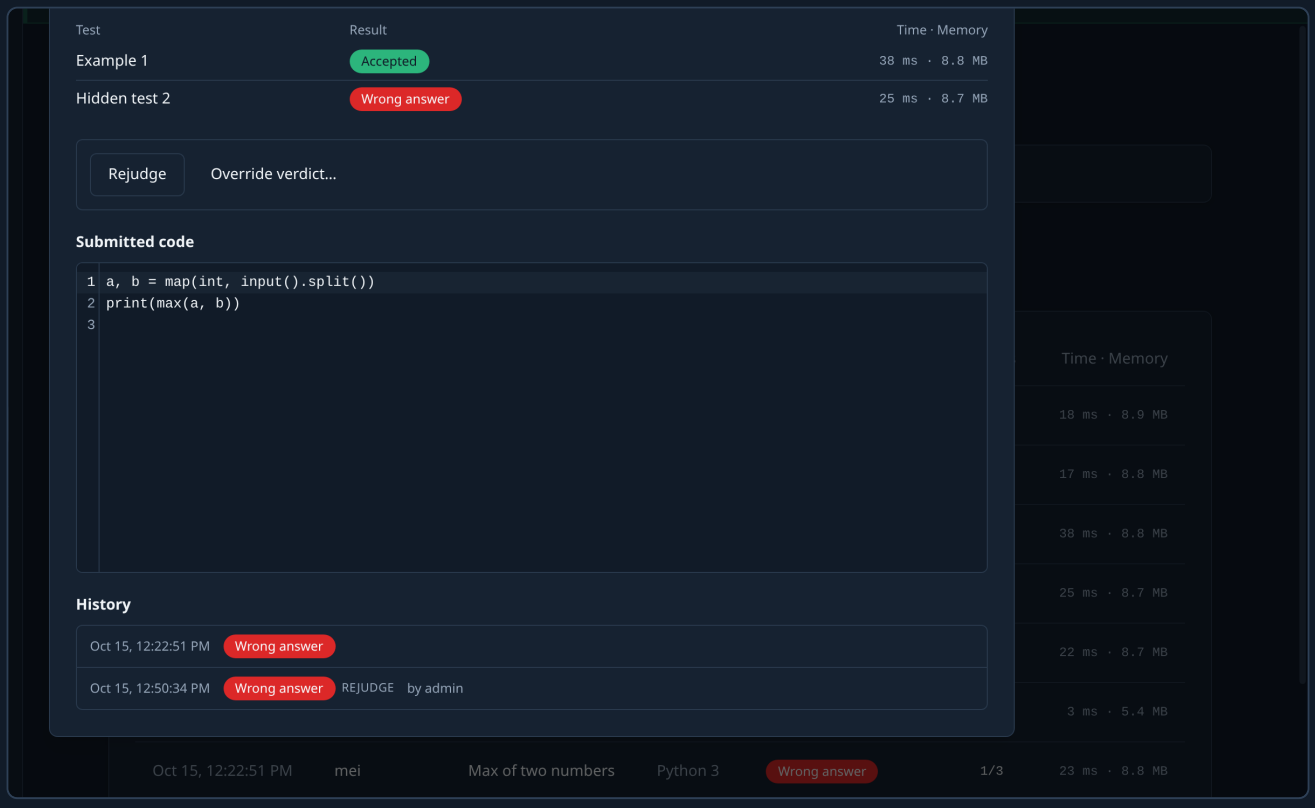
The time limit competitors see is per language — each language's multiplier (set by the administrator) is applied, so Python gets proportionally longer than C++.

Problem packages

Import problem package… on the Manage challenges page accepts a problem package in the open ICPC format — one problem or a contest zip — and creates each as a **draft**, with a list of things to check. Flagpost imports what it can judge faithfully (pass/fail problems with the standard output check) and **refuses the rest with the reason** — custom checkers, interactive or scored problems. Statements are converted from Markdown or LaTeX as best they can be, with the original attached. Most packages don't state a time limit: the dialog asks for one, and the package's accepted solution is pre-filled into the reference check so you can confirm it in one click.

Export problem packages writes every problem back out in the same format; a single problem's **Export package** sits in its Test cases section.

The Judging page



Every code submission as it's judged: the queue, per-test results, compiler messages and the submitted code.

Judging appears in the sidebar once the competition has a programming problem. Filter by **Problem** and **Result**, and **Include example runs** to see competitors' unscored trial runs. Open a submission for its tests, **Submitted code** and **History**, then:

- **Rejudge** one submission; or, with a problem selected, **Rejudge this problem…** — Every submission, Only accepted submissions (to check a fix broke nothing) or Only submissions that weren't accepted (after fixing a wrong test). Verdicts a judge set by hand are left alone.
- **Override verdict…** with a **Reason (shown to staff)**. It replaces the judge's verdict immediately and is recorded with your name.

The **newest verdict counts**: points, the scoreboard and the activity feed follow it, so a revoked solve takes its points back and a late acceptance scores from the original submission time. Competitors see Rejudged or Judge's decision on the entry, never the reason.

CHAPTER 4

Challenge instances

An **instanced challenge** hands each team (or player) its own running copy of a service, so nobody can trample another team's target or guess a shared one. It needs the **Challenge instances** module on for the competition and an administrator to have connected a container runtime (Admin → Site settings → Instances); until then nothing launches.

The Deployment section

Open an existing challenge and fill in **Deployment**:

- **Container image** — the image to run (from a registry the administrator's credential can reach).
- **Exposure** — TCP (competitors connect to a port), HTTP (subdomain + TLS) (each instance gets its own https address; needs the administrator to have set a base domain), or None.
- **Ports**, **Environment variables**, **Instances per team**, and resource overrides — **CPUs**, **Memory**, **PIDs**, **Lifetime** — which default to the site or competition values when blank (memory and PIDs take whole numbers).
- **Flag mode** — Shared flag (the challenge's own flag, same for everyone) or Unique per instance: a **Flag template** containing the random-token placeholder the field names. Each instance gets its own flag, injected into the container as the `FLAG` environment variable, and a submission is graded against the submitter's **own running instance**.

Set deployment saves it; **Remove** stops the challenge offering instances and reaps any running ones. A unique-flag challenge needs no static flag and can be published as usual.

> **UNIQUE FLAGS CATCH SHARING**
>
> With Unique per instance, submitting another team's flag is wrong — and quietly recorded as **flag sharing**. There's no automatic penalty and competitors aren't told; route the flag-sharing trigger into an automation (chapter 7) so staff hear about it, then decide.

Running instances

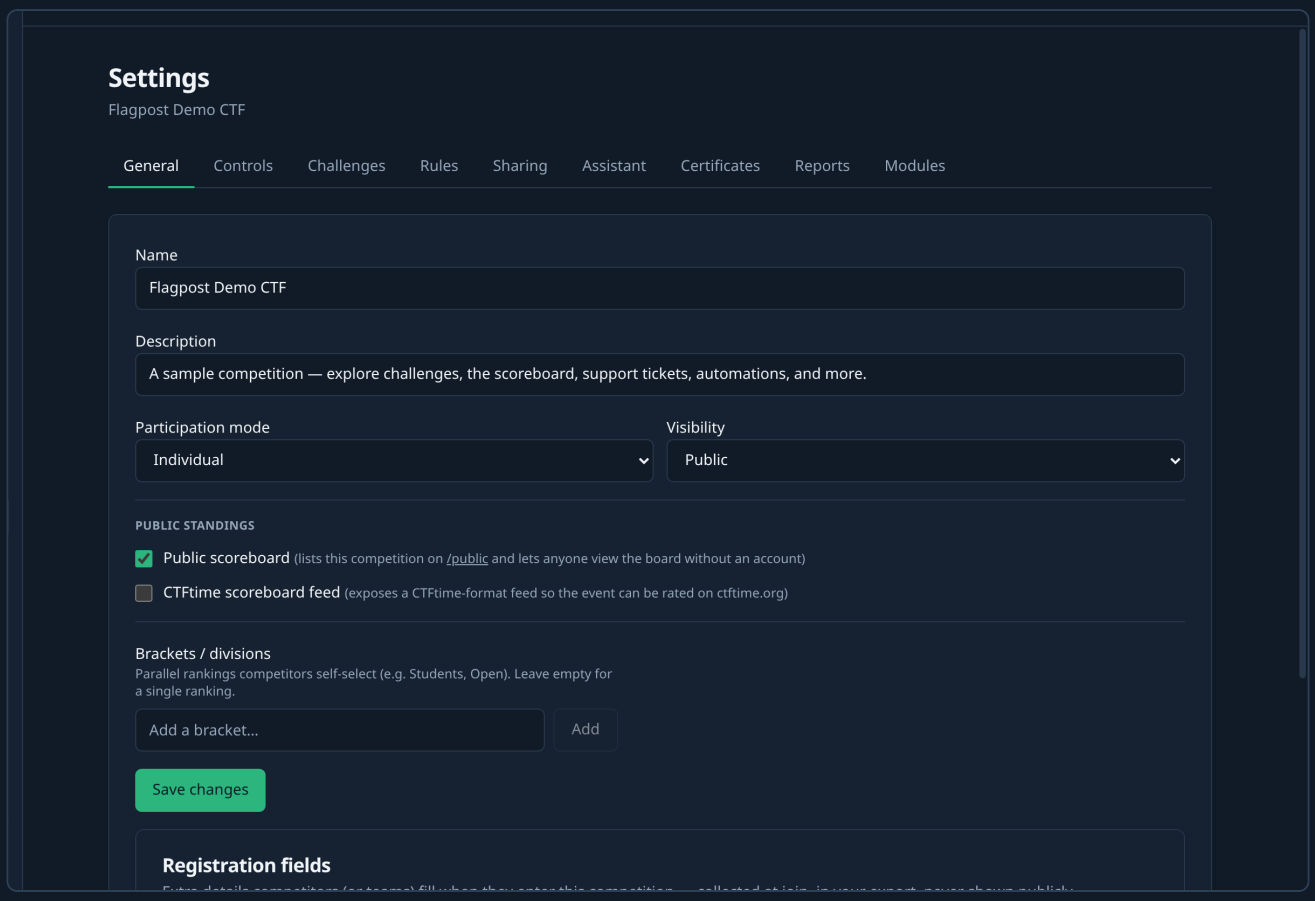
Instances appears in the sidebar once instancing is set up site-wide or the competition has deployments of its own; it lists every live instance — challenge, team or user, status, and when it expires — updating live. **Kill** destroys one immediately; the subject can launch again. The dashboard's **Instance health** section summarises instances by status with recent failures.

Competitors launch only while the competition is running (and, in team mode, once on a team); staff who can manage instances may test-launch at any time. A launch can take a while for a large image — pre-pull images on the instance host, and ask your administrator about the per-competition caps if you expect a rush at kickoff.

CHAPTER 5

Configuring & running the event

Everything about the competition itself lives in **Settings**, in tabs: General, Controls, Challenges, Rules, Sharing, Assistant, Certificates, Reports and Modules (the last five appear with the matching module or grant).



General holds identity, visibility, the public scoreboard and divisions, with Registration fields below; Controls holds the status, pause and freeze levers plus the schedule.

Before the event

- **General** — name, description, team vs individual mode (with **Max team size**), **Visibility** (Private or Public), **Public scoreboard** (lists the event on the site's spectator page), the **CTFtime scoreboard feed**, and **Brackets / divisions** — parallel rankings such as Students and Open. Below the form, **Registration fields** adds questions competitors or team captains answer on joining — text, choose-one or checkbox, optionally required — collected for your **Export CSV** and never shown publicly.
- **Challenges** — categories, difficulty tiers and tags; the **multiple-choice guess limit** and **wrong-guess penalty**; and whether competitors are asked to rate challenges.
- **Rules** — a per-competition override of the site's rules and code of conduct. Leave it empty to use the site document; tick **Display only** to show rules without requiring acceptance. Saving a new or changed override re-prompts every participant (**Save and re-prompt**).

- **Sharing — invite links.** **New link** mints a Join link (which, with the permanent invite code, is the only self-serve way into a private competition) or a View link to the public spectator board, each with an optional label, expiry and maximum joins; **Revoke** kills a link for good. Join links honour the registration window and stop admitting once the event ends.
- **Modules** — a tile for each of the eight optional features, with what it does and an on/off switch: AI assistants, Analytics, Automations, Certificates, Challenge instances, Code judge, Feedback, Reports. All start on; switch off what you won't use. AI, instances and the code judge also need the administrator's site setup before they do anything; their tiles say Needs site setup until then. The always-on core modules are listed below, folded away.

The status gate

A competition is **not started** → **running** → **ended**. Competitors can open challenges and submit only while it's running — before and after they see a closed notice (the final scoreboard stays visible), while staff retain full access. The **Schedule** at the bottom of Controls — Starts, Ends, Registration opens, Registration closes — drives the transitions and saves with the form; **Start competition** / **Stop competition** (and **Re-open**) override it on the spot.

Mid-event levers (Controls)

- **Gameplay pause** — closes flag submissions, code submissions and hint reveals during an incident. The Challenges page shows a paused banner; post an announcement too. Staff can still submit, to verify a fix.
- **Scoreboard freeze** — stop standings updates near the finale, in the app and on the public board alike. Solves keep counting; staff can switch the Scoreboard to **Show live standings**; unfreeze at your ceremony. While frozen, your dashboard stops updating on its own — reload it for current figures. **Freeze board** is also on the Scoreboard page.

> **DRY-RUN BEFORE DOORS OPEN**
>
> While the competition is not started, open every challenge and download every file. Rehearse the flags in a clone (ask an administrator) — staff solves on the live event are real (chapter 1). And save before switching competitions: unsaved Settings edits are discarded.

CHAPTER 6

Supporting competitors

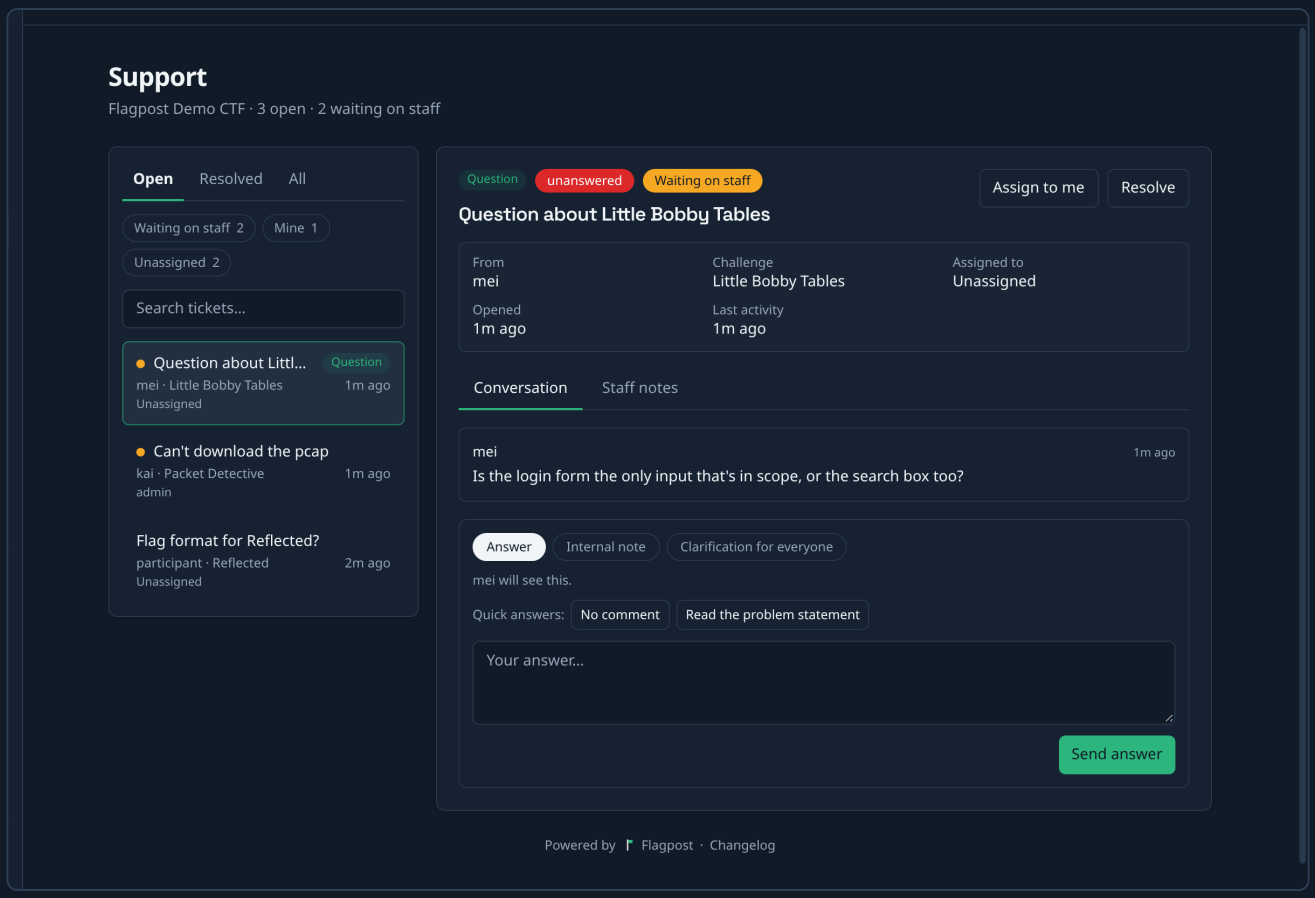
Announcements

New announcement on the dashboard reaches everyone in the competition — live as a banner on every open session, then in the archive and the bell. Three controls matter:

- **Urgency** — Announcement sits in the banner briefly; Important stands out; Urgent stays until dismissed and reaches even competitors who muted announcements. Use it for incidents.
- **Send to** — everyone, specific teams, or specific competitors.
- **Schedule for later** — pick a **Publish time** and it stays a private draft until then. Pending posts wait in the dashboard's **Scheduled announcements** card (Reschedule, Publish now, Cancel) — queue the hint drops and challenge waves before the event starts.

Announce before competitors notice, not after the tickets arrive.

The support queue



Tickets are live conversations — replies stream in real time on both sides, and staff have their own tools alongside.

Competitor tickets arrive in **Support:** the queue on one side, the open ticket beside it (on a phone, one after the other), with the open count on your dashboard. **Waiting on staff**, **Mine** and **Unassigned** narrow the queue, and a dot marks a ticket whose competitor wrote last. Each ticket has its own link, so you can send one to another judge.

On a ticket, **Assign to me** and **Resolve** sit at the top, apart from what you're writing. Below the conversation, choose what you're sending — **Reply**, an **Internal note** only staff see, or, on a ticket about a challenge, a **Clarification for everyone** — and the box changes colour to match. **Staff notes** is its own tab: a live pad for staff only. Competitors see A judge is looking at this ticket while you have it open.

Working principles: **acknowledge fast, resolve second**; **reproduce before you rule** ("the flag won't submit" is a claim — check it in a safe place); if a challenge really is broken, **pause or unpublish it**, announce, fix, then consider the scoring impact; and resolve tickets so the queue reflects reality.

Questions and clarifications

On a programming problem, competitors can **Ask the judges** privately. The question lands in Support tagged **Question** — one message you answer with **Send answer** (or a quick answer: No comment, Read the problem statement); they can't reply, only ask again. If the answer helps everyone, choose **Clarification for everyone**: edit The question, as everyone will see it — strip anything that gives away who asked or how they're solving it — and **Send to everyone**. The same option works on any ticket about a challenge.

You can also **Post a clarification** straight from a challenge, and **Withdraw** one later: it stays listed, marked withdrawn, with its answer removed everywhere. Clarifications only go out while the competition is running and the challenge is visible.

Guess resets and awards

When a competitor legitimately exhausts their multiple-choice guesses, open the challenge in the editor: **Guesses** offers Reset for selected or Reset for everyone — history is kept. In an individual-mode competition, **Participants** → **Create award** grants a titled award with points (zero for a badge, negative to deduct); in team mode, credit a team through an automation's Create award or Update score action.

> **FAIRNESS TRAVELS IN PAIRS**
>
> Anything you grant one team — a reset, a nudge, extra time on a target — be prepared to grant every team in the same situation. The dashboard's **Moderation feed** shows recent score changes and deletions, and the administrator's event log records everything; post-event write-ups read better when the answer to "why them?" is "same as everyone".

CHAPTER 7

Watching the event

Reading the room

The dashboard's sections are your event telemetry:

- **Challenge health** — solves vs. attempts per challenge. Many attempts and no solves can mean good and hard — or broken. **Unsolved challenges** lists the ones nobody has cracked.
- **Recent solves** — the live stream. A burst right after a hint tells you the hint priced correctly; a first blood minutes after release tells you it didn't.
- **Brute-force watch** — the teams submitting the most wrong flags; **Team activity** shows who's gone quiet.
- **Support queue** — an open count trending up is the earliest warning of an infrastructure or clarity problem.

On the **Scoreboard**, staff can **Freeze board** / **Unfreeze board**, and in a divisioned event assign each competitor's **Division** from the table. While it's frozen, **Show live standings** switches to the real standings, still updating live. It starts on the frozen board each time, so don't project the page after switching. The public board has a **Venue mode** for projectors.

Optional instrumentation

If the modules are on for your competition (Settings → Modules):

- **Analytics** — Overview: a per-challenge table (solves, completion, average time, attempts, hints, tickets, rating), the teams or competitors table, and insight cards (least solved, most attempted, most tickets, most first bloods). Submissions: a filterable browser of every submission with **Export CSV**.
- **Feedback** — challenge ratings (once enabled in Settings → Challenges) and **surveys**: **New survey** with rating, text and multiple-choice questions, **Open for responses**, **Results** and **Export CSV**.
- **Automations** — rules built as When this happens → And these are all true → Do this: announce or award on first blood, release a hint or freeze the board at a set time remaining, alert staff on flag sharing. Actions include hints, unlocks, announcements, awards and score updates, notifications, email and webhooks. **My rules** holds personal notification rules.
- **Assistant** (Settings) — the competitor AI assistant's **Availability**, **Guidance level** (Platform only, Conceptual or Guided — never full solutions) and whether it can see **Challenge details**. The sidebar's **AI transcripts** is how you verify it's behaving; staff also get a read-only **Organiser assistant**.
- **Certificates** (Settings) — a visual designer; **Recipients** (All participants or Only participants who scored); **Release timing** (manual, at the end, or a set time after); **Export all as ZIP**; and a reusable template. Save the design before releasing — the saved one is what ships.
- **Reports** (Settings) — once the competition has ended, **Generate report**: an audience preset (Executive, Technical, Full) or your own sections, as PDF and/or HTML, kept in a version history.

Closing out

When the schedule ends (or you press Stop), submissions close and the board settles. Unfreeze the scoreboard at your ceremony, release certificates, generate the report, and skim the feedback while memories are fresh. Export the challenge set for next year — the ctfcli zip (minus flags) and **Export problem packages** — and hand archiving and backups to your administrator.

The deeper operational reference — deployment, scaling, integrations — lives at **docs.flagpost.io**.

CHAPTER 8

FAQ

What do competitors see of a draft?

Nothing — no card, no count, no scoreboard trace. Drafts (including unpublished-again challenges) exist only for staff, so you can build and repair openly during a live event.

I edited a live challenge — what happens to points already scored?

Description, connection info, hints and attachments apply immediately. On a **static** challenge, existing solves keep the value they scored and future solves earn the new one; on a **dynamic** challenge every solver is re-valued as the decay moves. If a mistake cost competitors points, compensate with an award or score adjustment rather than retro-editing values.

A test case was wrong. Now what?

Fix it, then on the **Judging** page pick the problem and **Rejudge this problem…** → Only submissions that weren't accepted. Newly accepted submissions score from their original submission time, so first blood and ordering come out right. Run the reference check again afterwards — it tells you when it's stale.

Where's the Judging page?

It appears in the sidebar only when the competition's Code judge module is on and the competition has at least one programming problem (drafts count). If Programming isn't selectable in the challenge editor, either the competition's Code judge module is off (Settings → Modules) or the administrator hasn't finished the judge setup — the editor says which.

Can I grant bonus points?

In an **individual-mode** competition, yes: Participants → **Create award** (points can be zero or negative). In a team competition, awards and score changes go through an automation's Create award / Update score action.

A team legitimately burned all their guesses.

Open the challenge in the editor — its **Guesses** section restores attempts for one team/competitor or for everyone. History is kept.

Pause or stop — which do I want?

Pause for incidents: flag and code submissions and hint reveals close, the Challenges page shows a banner, the schedule keeps its dates, and staff can still submit to verify a fix. **Stop** ends gameplay. If an incident ate meaningful time, pause, fix, extend the end time in Controls, then resume.

When can I safely test challenges myself?

Opening and downloading is always safe. **Submitting flags on the live competition is not** — staff solves take first blood and decay dynamic values (and need a team in team mode). Rehearse in a clone; for programming problems, Run examples and the reference check never score.

Import said it skipped or couldn't import some challenges.

The ctfcli importer **skips** challenges whose title already exists and never overwrites; static flags aren't in the export, so re-add them. The problem-package importer lists Skipped (title exists) and Couldn't import with the reason — a custom checker, an interactive or scored problem — which Flagpost can't judge faithfully. Imported problems arrive as drafts: review the converted statement and confirm the time limit with the reference check.

Why can't a competitor join my private competition?

The invite code or a **join link** (Settings → Sharing) is the only self-serve way in. Join links also honour the registration window in Controls, and stop working once the event ends.